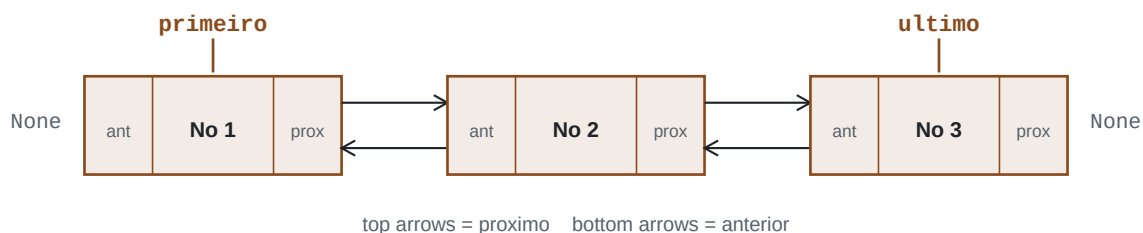


Padaria - Code Reference

Bakery Management System (ISUTC, Algoritmos e Estruturas de Dados, TP1). Every class, its variables, and what each function does and how.

Overview

`main.py` calls `main()` in `src/app.py`, which opens three screens in order: **TelaInicial** (splash) -> **TelaLogin** -> **AplicacaoPadaria** (main window). Products and sales are stored in two **doubly linked lists** (`ListaLigada` and `ListaVendas`). The interface only reads the form, converts the values, calls a list method and redraws the tables. Nothing is saved to disk: every run starts with sample data.



Each node points to the node before it (**anterior**) and after it (**proximo**). The list keeps **primeiro** (head), **ultimo** (tail) and **tamanho** (size). Traversal loops use a cursor called **actual**.

1. Data structures (src/estruturas)

class No (lista_produtos.py)

A node of the product list: one product plus links to its neighbours.

Variable	Type	Meaning
<code>codigo</code>	int	Unique product code.
<code>nome</code>	str	Product name.
<code>categoria</code>	str	Category (e.g. Pão, Bolo, Doce).
<code>preco</code>	float	Unit price (MT).
<code>quantidade</code>	int	Units in stock.
<code>validade</code>	str	Expiry date (dd/mm/aaaa).
<code>anterior</code>	No None	Previous node; None for the first node.
<code>proximo</code>	No None	Next node; None for the last node.

class ListaLigada (lista_produtos.py)

Doubly linked list holding all products.

Variable	Type	Meaning
<code>primeiro</code>	No None	First node (head). None when the list is empty.
<code>ultimo</code>	No None	Last node (tail). Allows adding at the end without walking the list.
<code>tamanho</code>	int	Number of nodes. Updated on every insert and delete.

Functions

Function	What it does	How
<code>__init__()</code>	Creates an empty list.	<code>primeiro = ultimo = None</code> , <code>tamanho = 0</code> .

Function	What it does	How
cadastrar(codigo, nome, categoria, preco, quantidade, validade)	Adds a product at the end.	Rejects a repeated code (ValueError). Creates a No; if the list is empty it becomes primeiro and ultimo, otherwise it is linked after ultimo (novo.anterior = ultimo, ultimo.proximo = novo) and becomes the new ultimo. O(n) because of the duplicate check; the insert itself is O(1).
buscar_por_codigo(codigo)	Finds a product by code.	Walks from primeiro following proximo; returns the node or None. O(n).
buscar_por_um_atributo(atributo, valor)	All products where one attribute equals a value.	Walks the list, reads the attribute with getattr, compares both sides as trimmed, lowercase text (so 'pão' matches 'Pão'). Returns a Python list of nodes.
buscar_por_dois_atributos(a1, v1, a2, v2)	Products matching two attributes at once.	Same walk; keeps a node only if both comparisons are true (AND).
alterar_por_codigo(codigo, novos_dados)	Edits a product.	Finds the node by code (ValueError if missing). For each key/value in the dict, sets the attribute with setattr, skipping empty values ("" or None) so blank fields keep their old value.
_desligar(no)	Internal helper: removes a node from the chain.	Links the neighbours to each other in both directions. If the node was first, primeiro moves to the next one; if it was last, ultimo moves to the previous one. Clears the node's links, decrements tamanho, returns the node. O(1).
eliminar_por_posicao(posicao)	Deletes the node at a position (1 = first).	IndexError if outside 1..tamanho. Walks from primeiro (forward) when the position is in the first half, otherwise from ultimo (backwards via anterior), then calls _desligar.
eliminar_por_codigo(codigo)	Deletes the product with that code.	ValueError if the list is empty or the code is not found; otherwise buscar_por_codigo + _desligar.
listar_todos()	All products in insertion order.	Walks from primeiro and collects every node into a Python list.
listar_por_criterio(atributo, valor)	Products matching a criterion.	Delegates to buscar_por_um_atributo.
listar_ordenado(atributo, decrescente=False)	Products sorted by an attribute.	Copies the nodes with listar_todos, then bubble sort: repeated passes swapping neighbours that are out of order. O(n ²). The linked list itself is not reordered.

class NoVenda (lista_vendas.py)

A node of the sales list: one sale.

Variable	Type	Meaning
codigo_produto	int	Code of the product sold.
nome_produto	str	Product name at the time of the sale.
quantidade	int	Units sold.
preco_unitario	float	Price per unit at the time of the sale.
total	float	quantidade × preco_unitario, computed in the constructor.
data	str	Timestamp of the sale (dd/mm/aaaa hh:mm), from datetime.now().
anterior	NoVenda None	Previous sale.

Variable	Type	Meaning
<code>proximo</code>	NoVenda None	Next sale.

class ListaVendas (lista_vendas.py)

Doubly linked list holding every sale, in the order they happened.

Variable	Type	Meaning
<code>primeiro</code>	NoVenda None	First sale (head).
<code>ultimo</code>	NoVenda None	Last sale (tail).
<code>tamanho</code>	int	Number of sales.

Functions

Function	What it does	How
<code>__init__()</code>	Creates an empty list.	<code>primeiro = ultimo = None</code> , <code>tamanho = 0</code> .
<code>registrar_venda(codigo_produto, nome_produto, quantidade, preco_unitario)</code>	Records a sale at the end.	Creates a NoVenda and links it after ultimo (same as cadastrar). O(1). Returns the new node.
<code>listar_todas()</code>	All sales in order.	Walks from primeiro, collects the nodes.
<code>total_vendas()</code>	Total revenue.	Walks the list summing each node's total.
<code>total_quantidade()</code>	Total units sold.	Walks the list summing each node's quantidade.

2. Interface (src/interface)

All screens use Tkinter. Each screen receives the main window (root) and a callback that it calls when it is done, so the screens do not need to know about each other.

class TelaInicial (tela_inicial.py)

Splash screen: background image, system name and an "Acessar" button.

Variable	Type	Meaning
CAMINHO_IMAGEM_FUNDO	Path (module)	Path to assets/imagens/fundo.png, built from this file's location.
LARGURA	int (class)	Window width: 640 px.
ALTURA_IMAGEM	int (class)	Height of the image area: 320 px.
ALTURA_RODAPE	int (class)	Height of the footer with the button: 90 px.
root	tk.Tk	Main window.
ao_acessar	function	Callback run when "Acessar" is clicked (opens the login).
imagem_fundo	tk.PhotoImage	The background image. Kept as an attribute so Python does not garbage-collect it (it would disappear from the screen).

Functions

Function	What it does	How
<code>__init__(root, ao_acessar)</code>	Builds the splash screen.	Sets title/size, draws the image and title text on a Canvas, adds a dark footer with a styled "Acessar" button centred with place().
<code>_acessar()</code>	Leaves the splash screen.	Destroys every widget in the window and calls <code>ao_acessar()</code> .

class TelaLogin (tela_login.py)

Login screen. Only users listed in UTILIZADORES get through.

Variable	Type	Meaning
root	tk.Tk	Main window.
ao_autenticar	function	Callback run with the user's record after a successful login.
entrada_utilizador	ttk.Entry	Username field.
entrada_senha	ttk.Entry	Password field (shows *).
label_erro	ttk.Label	Red label that shows "Utilizador ou senha inválidos."

Functions

Function	What it does	How
<code>__init__(root, ao_autenticar)</code>	Builds the login form.	Title, two fields, error label and "Entrar" button. Enter in the username jumps to the password; Enter in the password tries to log in.
<code>_tentar_login()</code>	Checks the credentials.	Looks up the username in UTILIZADORES and compares the password. Wrong: shows the error and clears the password. Right: destroys the widgets and calls <code>ao_autenticar(registo)</code> .

class AplicacaoPadaria (aplicacao.py)

Main window after login: top bar, sidebar with operations, product form and two tabs (Registos / Vendas).

Variable	Type	Meaning
ATRIBUTOS	list (class)	Product fields in order: codigo, nome, categoria, preco, quantidade, validade.
ATRIBUTOS_LABEL	dict (class)	Display label for each field (e.g. preco -> "Preço").
lista	ListaLigada	All products.
lista_vendas	ListaVendas	All sales.
root	tk.Tk	Main window.
perfil	dict	Logged-in user's record (senha, tipo, nome).
e_dono	bool	True if the profile is "Dono da Padaria".
ao_terminar_sessao	function	Callback run on logout (restarts from the splash).
container	ttk.Frame	Holds the sidebar (left) and content area (right).
entradas	dict	Form fields: attribute name -> ttk.Entry.
abas	ttk.Notebook	The tab control.
tab_registros / tab_vendas	ttk.Frame	The two tabs.
tabela_registros	ttk.Treeview	Products table.
tabela_vendas	ttk.Treeview	Sales table.
frame_resumo_vendas	ttk.Frame	Bar under the sales table with totals and a button.
label_total_qtd / label_total_vendas	ttk.Label	Show total units sold and total revenue (MT).

Setup and layout

Function	What it does	How
<code>__init__(root, perfil, ao_terminar_sessao)</code>	Starts the main window.	Creates both lists, stores the profile, sets e_dono, window size, loads sample data, builds layout.
<code>_construir_layout()</code>	Builds the whole window.	Top bar, then container with sidebar and content.
<code>_construir_barra_topo()</code>	Top bar.	Label with user name and profile; "Sair" button.
<code>_terminar_sessao()</code>	Logout.	Asks for confirmation, destroys the window, calls ao_terminar_sessao.
<code>_construir_sidebar()</code>	Operation buttons.	Loops over (text, method) pairs creating a button each. Skips any operation in OPERACOES_RESTRIAS_AO_DONO when the user is not the owner.
<code>_construir_area_conteudo()</code>	Right-hand area.	Form on top, tabs below.
<code>_construir_formulario(parent)</code>	Product form.	One label + Entry per attribute in a 2 x 3 grid; saves the entries in self.entradas.
<code>_construir_abas(parent)</code>	The two tabs.	Registros: Treeview with one column per attribute + scrollbar. Vendas: sales Treeview, totals labels and "+ Registrar Venda" button; fills the sales table.
<code>_popular_exemplo()</code>	Sample data.	Adds 4 products and 3 sales.

Helpers

Function	What it does	How
<code>_atualizar_tabela_registros(produtos=None)</code>	Redraws the products table.	Clears the Treeview and inserts the given products, or all of them when none are given.
<code>_atualizar_tabela_vendas()</code>	Redraws the sales table.	Clears and refills it from <code>lista_vendas</code> and updates both total labels.
<code>_ler_formulario()</code>	Reads the form.	Returns {attribute: trimmed text}.
<code>_limpar_formulario()</code>	Clears the form.	Empties every Entry.
<code>_preencher_formulario(produto)</code>	Fills the form with a product.	Clears it, then inserts each field.
<code>_exige_dono()</code>	Permission check.	If the user is not the owner, shows "Acesso restrito" and returns True (the caller stops).
<code>_pedir_atributo_valor(titulo)</code>	Popup: pick attribute + type value.	Modal window (grab_set + wait_window) with a Combobox and an Entry; returns (atributo, valor), or (None, None) if closed.
<code>_pedir_valor_simples(titulo, label_texto)</code>	Popup: type one value.	Same modal pattern with one Entry; returns the text or None.
Actions (sidebar buttons)		
Function	What it does	How
<code>acao_buscar_produto()</code>	Search by code.	Popup asks the code; <code>buscar_por_codigo</code> ; fills the form and shows only that product.
<code>acao_cadastrar() [owner]</code>	Add product.	Reads the form, converts <code>codigo/quantidade</code> to int and <code>preco</code> to float, calls <code>lista.cadastrar</code> , refreshes the table. Invalid numbers or a repeated code show an error.
<code>acao_buscar_um_atributo()</code>	Search by 1 attribute.	<code>_pedir_atributo_valor</code> + <code>buscar_por_um_atributo</code> ; shows results or "Nenhum produto encontrado".
<code>acao_buscar_dois_atributos()</code>	Search by 2 attributes.	Popup with two attribute/value pairs; <code>buscar_por_dois_atributos</code> .
<code>acao_alterar() [owner]</code>	Edit product.	Requires a code in the form; builds a dict from the other fields (empty = keep) and calls <code>alterar_por_codigo</code> .
<code>acao_eliminar_posicao() [owner]</code>	Delete by position.	<code>_pedir_valor_simples</code> ; <code>eliminar_por_posicao</code> ; shows the removed product's name.
<code>acao_eliminar_codigo() [owner]</code>	Delete by code.	<code>_pedir_valor_simples</code> ; <code>eliminar_por_codigo</code> .
<code>acao_listar_todos()</code>	Show all.	Refreshes the table and selects the Registos tab.
<code>acao_listar_criterio()</code>	List by criterion.	<code>_pedir_atributo_valor</code> + <code>listar_por_criterio</code> .
<code>acao_listar_ordenado()</code>	List sorted.	Popup with attribute Combobox and "Ordem decrescente" checkbox; <code>listar_ordenado</code> .
<code>_registar_venda()</code>	Record a sale (sidebar and Vendas tab).	Popup asks code and quantity. Checks: positive integers, product exists, enough stock. Then subtracts the quantity from stock, calls <code>registar_venda</code> , refreshes both tables, shows the total.

3. Module-level code

config.py

Variable	Type	Meaning
UTILIZADORES	dict	username -> {senha, tipo, nome}. Two users: dono / dono123 (Dono da Padaria) and funcionario / func123 (Funcionário).
OPERACOES_RESTRITAS_AO_DONO	set	Button texts only the owner sees: Cadastrar, Alterar (por código), Eliminar por posição, Eliminar por código.

app.py and main.py

Function	What it does	How
main()	Program start.	Creates the Tk window, calls <code>iniciar_aplicacao</code> , starts <code>mainloop()</code> .
iniciar_aplicacao(root)	Chains the three screens.	Defines two inner callbacks: <code>ao_acessar</code> opens <code>TelaLogin</code> , <code>ao_autenticar(perfil)</code> opens <code>AplicacaoPadaria</code> . Then shows <code>TelaInicial</code> with <code>ao_acessar</code> .
ao_terminar_sessao()	Restart after logout.	Creates a new Tk window and calls <code>iniciar_aplicacao</code> again.
main.py	Entry point.	<code>python main.py -> src.app.main()</code> .